

TOP: Time Optimization Policy for Robust and Accurate Humanoid Standing Manipulation

Abstract—Humanoid robots are expected to perform diverse manipulation tasks, yet this relies on a standing controller that is both robust and precise. Existing methods often fail to accurately control high-dimensional upper-body motions or to guarantee stability when these motions are fast. We propose a Time Optimization Policy (TOP) that enables humanoid robots to achieve balance, precision, and time efficiency simultaneously by optimizing the time trajectories of upper-body motions, rather than solely enhancing lower-body disturbance rejection. Our framework integrates three key components: (i) a variational autoencoder (VAE) to encode motion priors and improve coordination between upper and lower body, (ii) a decoupled controller consisting of an upper-body PD controller for precision and a lower-body RL controller for robustness, and (iii) joint training of the controller with TOP to mitigate destabilization caused by fast upper-body motions. We validate our approach in both simulation and real-world experiments, demonstrating superior stability and accuracy in humanoid standing manipulation tasks. The project page can be found at <https://anonymous.4open.science/w/top-258F/>.

I. INTRODUCTION

Humanoid robots are the most potential embodied agents for the purpose of liberating human-level labors, as they are designed to perform anthropomorphic motions and various whole-body loco-manipulation tasks, including industrial parts assembly, home service, etc.[1]. Their anthropomorphism naturally makes them more suitable than other specific robots to interact with environments, objects and humans to complete various physical tasks. Although rapid growth has been achieved in the field of humanoid robots[2], it remains a challenge to execute various intricate tasks while maintaining balance and precision simultaneously due to the intrinsic instability characteristic of humanoid robot.

Existing methods can be broadly divided into two paradigms: whole-body controllers[3, 4, 5] and upper and lower-body decoupled controllers[6, 7]. Traditional whole-body controllers, such as model predictive control (MPC), are capable of generating precise motions[3], but maintaining stability and robustness in real world remains a significant challenge. Whole-body RL-based controllers offer improved dynamic robustness[8, 9], but often struggle to accurately track complex reference trajectories, particularly involving high-dimensional upper-body joints[10], and are prone to overfitting to suboptimal behaviors or generating unpredictable actions. To mitigate these limitations, a decoupled control architecture has been proposed[6], wherein the high-dimensional upper body is controlled by a PD controller to ensure precise trajectory tracking, while the lower body is governed by an RL policy to provide robust balance against external perturbations. This decoupled controller has shown promise in enhancing

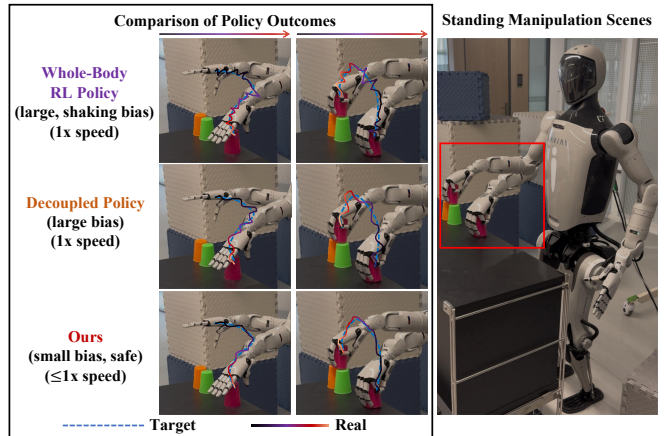


Fig. 1: Illustration of different methods. **A**: Whole-body RL policy, but it will be ill-suited to control high-dimensional upper-body joint, which cause large and shaking bias from target trajectory. **B**: Decoupled policy avoid shaking bias by using PD controller for upper-body, but still lack the consideration about momentum caused by rapid upper-body motions, remaining large bias. **C**: Our method adjust timestamp of motions aiming to reduce the impact of momentum and making standing safer, which gain smaller bias but need more time to achieve the goal.

both the stability and precision required for standing manipulation tasks[7].

However, all the aforementioned methods tend to execute upper-body motions without fully considering the robot’s actual execution capabilities, as shown in fig. 1. This oversight often neglects the dynamic consequences of momentum changes caused by fast upper-body motions, which can lead to instability, loss of balance, or even collisions with the environment. A key issue lies in the momentum introduced by upper-body motions: fast upper-body motions may destabilize the robot and impact the tracking precision, while slower motions can certainly reduce momentum changes and improve stability and accuracy, but sacrifice time efficiency. In other words, whether the reference trajectory is generated by teleoperation[2], VLA[11] or other planners[12], determining the appropriate motion speed remains a tricky problem in whole-body standing manipulation scenarios.

Therefore, in this paper, we propose a novel learning-based framework to improve lower-body stability and upper-body motion precision by using **TOP** to optimize the timestamps of upper-body motions, minimizing momentum-induced balance disturbances. Our approach aims to strike a balance between stability, accuracy, and time efficiency by optimizing the time

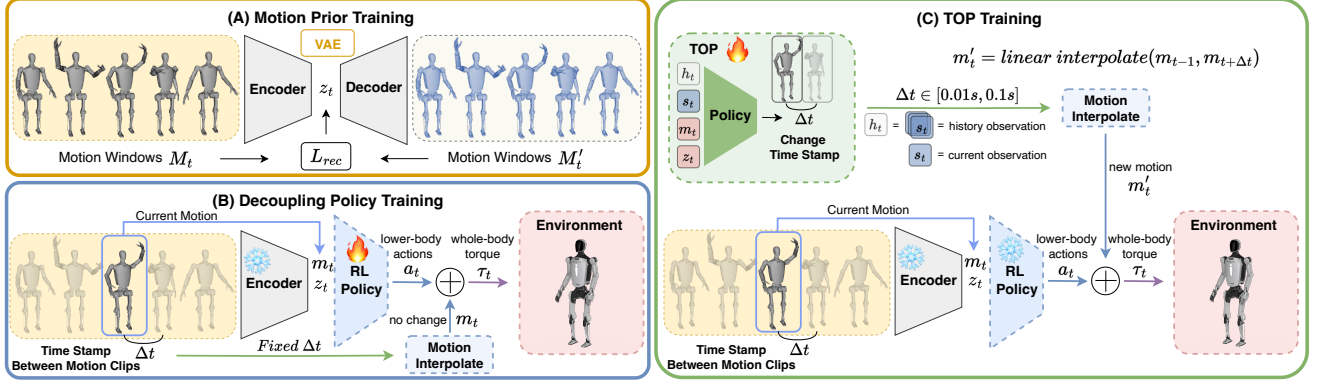


Fig. 2: The overall architecture. (A) Training a latent code z_t based on VAE structure to represent diverse upper-body motions. (B) A balance RL based policy to control the robot stay still, while upper-body use PD controller to execute current motion m_t , with a fixed time stamp Δt between motions. (C) Then we train the TOP to optimize the timestamp between the motion clips to reduce the speed of motions and the impact of the momentum changes, which will slow down the current motion m_t and execute new motion m'_t calculated by $\text{linear_interpolate}(m_{t-1}, m_{t+\Delta t})$.

trajectory of motions to determine motion speed. This reduces the effect of fast upper-body motions on overall momentum and enhances both balance and execution efficiency, while minimizing disturbances from lower-body shifts that could affect upper-body tracking. The framework consists of three stages: first, a structured representation of diverse upper-body motions is learned to capture motion clip features; second, training a RL-based lower-body policy to maintain balance while the upper body executes motion clips via a PD controller; finally, training TOP method via supervised reinforcement learning to optimize the timestamps between motion clips. This alleviates balance challenges caused by fast upper-body motions that often exceed the limits of balance controllers, preventing falls and improving performance in standing manipulation tasks. We validate our approach in both simulation and real-world environments, demonstrating its versatility and effectiveness. Our contributions are as follows:

- propose a novel framework for humanoid standing manipulation that realizes stable, accurate, and time-efficient execution of complex upper-body motions and manipulation tasks.
- As a key component of this framework, we introduce **TOP** (Time Optimization Policy), a supervised reinforcement learning module that optimizes motion timestamps to jointly enhance accuracy, stability, and efficiency.
- our method is tested both in simulation and the real world that can generalize well to practical standing manipulation tasks, which demonstrates the adaptability and efficiency of our approach.

II. RELATED WORKS

A. Whole-body Balance Controller

The primary challenge of humanoid robots in achieving diverse rich and precision motions is the whole-body controller that tracks accurate upper-body motions and stabilizes the entire body in real time[1]. Previously, different attempts have been made in dynamics modeling and control[13, 14], but these methods sacrifice precise models and fail to track large-scale motions. Moreover, the dilemma of balancing computationally efficiency and model complexity restricts its practical application[3].

tionally efficiency and model complexity restricts its practical application[3].

Data-driven approaches are gradually taking the lead in this field, especially reinforcement learning combined with some sophisticated methods [15, 16]. Zhang et al. [17] propose a method of whole-body locomotion by leveraging human motion references through imitation and optimization techniques, but it is difficult to generalize it to more whole-body motions. Fu et al. [9] introduce a framework based on transformer structures for humanoid robots to shadow and imitate human motions, improving natural interaction and adaptability. He et al. [18] successfully overcome the sim-to-real gap and train a universal and dexterous system for whole-body teleoperation and learning. However, these works focus on whole-body control with smaller arm DoFs and do not pay attention to the impact of lower-body stability on upper-body motion accuracy.

To address the limitations of RL in precisely controlling high-dimensional upper-body joints, researchers adopt a decoupled control architecture that combines a PD controller for accurate upper-body motion execution with an RL-based lower-body controller to ensure robust stability. Cheng et al. [19] decouples the upper and lower-body controllers to obtain precision, but still lacks a measurement of the accuracy of upper-body motion and may lead to instability when the robot is standing still. Lu et al. [7] propose a decoupled controller with CVAE to represent upper-body motions, aiming to maintain balance and precisely control in standing manipulation. On their basis, as a comparison, our approach integrates upper and lower-body separation control with time optimization policy that adjust the timestamp of upper-body motion, ensuring to improve standing stability, motion precision and time efficiency.

B. Motion Representation Learning

Motions from human or robots have spatial-temporal features in high-dimensional spaces[20], and an effective representation to extract these complex spatial-temporal structures is highly needed[21]. Peng et al. [22] introduced adversarial training to learn motion priors, improving reinforcement policy efficiency and generalization. Hassan et al. [23]

present a similar adversarial imitation learning framework to generate realistic interactions by integrating simulation with data-driven motion synthesis. However, when faced with large-scale data, it is difficult for such methods to learn a universal strategy, failed with mode collapse[24].

Another common way to represent complex motion is to filter and extract key motion information to represent smooth temporal and spatial composition[25], but struggle with handling highly diverse or long-range motions. Recently, advances in generative models, like diffusion models[26] or variational autoencoder(VAE)[27], provided a good idea for representation of motions by creating a latent code to learn the distribution of multimodal motion sequences [4]. Tevet et al.[28] uses a transformer-based diffusion framework to generate natural human motions by directly predicting samples with geometric constraints. Li et al.[29] unifies human motion estimation and generation in a single framework, leveraging regression and diffusion to handle diverse conditions and tasks jointly. To provide a multimodal representation of upper-body motions, we decide to use VAE to extract a latent kinematic motion space and reconstruct them for training.

III. METHOD

A. Overview

Our framework as depicted in Figure 2. The motion prior models the multimodal distribution of upper-body motions to improve coordination between the upper- and lower-body controllers. In the decoupling controller training stage, we adopt a curriculum schedule to ease RL exploration and fix the timestamps of motion clips, using the motion prior to train a robust lower-body controller. In the TOP training stage, we introduce supervised reinforcement learning to optimize the time stamps, with initial guesses provided by supervised learning. Action chunking [12] is employed to further smooth the time stamps, as detailed in III-D.

B. Extract Motion Priors

To improve awareness of the lower-body controller for upper-body past and future motions, we provide prior knowledge of upper-body motion, which is important for training more robust lower-body controllers [10, 19]. Specifically, we train an encoder-decoder pair to reconstruct the sequence of upper-body motions and incorporate the latent space as a representation in the state space of the lower-body control.

To represent diversity kinematic motions of a human or robot and capture more fine-grained level of motions, we use variational autoencoder (VAE) structure, which has already been shown as an effective motion representation and has the ability to learn the motion distribution and similarities of adjacent motions. We extract kinematic state of upper-body motions, consisting of joint position, velocity for a few past and future window frames. Our VAE structures include an encoder E and a decoder D , and the latent space is modeled as a multivariate Gaussian distribution $\mathbf{z}_t \in \mathbb{R}^{d_z}$.

$$\mathbf{m}_t = \{\mathbf{r}_t, \boldsymbol{\theta}_t, \mathbf{q}_t^{upper}, \dot{\mathbf{q}}_t^{upper}\}. \quad (1)$$

where $\mathbf{r}_t \in \mathbb{R}^3$ is the position of the base relative to the world frame. Since we only need to encode the upper-body motion, setting the $\mathbf{r}_t$ as the constant is natural. $\boldsymbol{\theta}_t$ is the orientation of the base frame, represents as 6D vector. The joint angles and velocities are given by $\mathbf{q}_t^{upper} \in \mathbb{R}^{n_j}$ and $\dot{\mathbf{q}}_t^{upper} \in \mathbb{R}^{n_j}$, where $n_j = 15$ includes two 7-dof arms and one waist joint.

Formally, we extract the past and future frames to consist motion windows of length $2W + 1$ from the distinct motion clips. In order to ensure the stability of the training of VAE networks, it is common to normalize the input data, and we use the mean and standard deviation of all motion clips in dataset to normalize our motion window frames, except orientation.

$$\mathbf{M}_t = \{\mathbf{m}_{t-W}, \dots, \mathbf{m}_t, \dots, \mathbf{m}_{t+W}\}. \quad (2)$$

The encoder of our VAE $E_\phi(\mathbf{z}_t|\mathbf{M}_t)$ maps the motion window to latent space $\mathbf{z}_t \in \mathbb{R}^{d_z}$, $d_z = 64$, and the sampled latent variable is then mapped back to input space by the decoder $D_\theta(\mathbf{M}'_t|\mathbf{z}_t)$. And we decide to use β -VAE[30] with the reconstruction loss as follows:

$$L_{rec}(\mathbf{M}_t, \mathbf{M}'_t) = \frac{1}{2W+1} \sum_{i=t-W}^{t+W} l_{rec}(\mathbf{m}_t, \mathbf{m}'_t) \quad (3)$$

$$l_{rec}(\mathbf{m}_t, \mathbf{m}'_t) = \|R(\boldsymbol{\theta}_t) - R(\boldsymbol{\theta}'_t)\| + \|\mathbf{q}_t - \mathbf{q}'_t\| + \|\dot{\mathbf{q}}_t - \dot{\mathbf{q}}'_t\| + \|\mathbf{p}_t - \mathbf{p}'_t\| \quad (4)$$

where the $R(\cdot)$ represents that computing rotation matrices for orientations using the Gram-Schmidt process, and because we already normalize the quantities, no relative weights are needed here. It is worth noting that the window size of $\mathbf{M}_t$ should be short enough to achieve motion generalization that the latent space can capture the features of primitive motion blocks that may appear in unseen motion sequences.

C. Training Decoupling Policy

We train a balance policy using a Legged Gym-based reinforcement learning framework, where PPO was used to update our lower-body policy. During training, the motion sequence is randomly chosen from the dataset at the beginning of a new episode, and retrieving the motion pair $(\mathbf{m}_t, \mathbf{z}_t)$ from Encoder $E_\phi(\mathbf{z}_t|\mathbf{M}_t)$. Then, we feed the motion pair to our lower-body policy to provide the instantaneous kinematic reference motions of upper-body and the past and future information, which helps the comprehension of the policy about disturbances caused by upper motions on balance.

We consider our balance lower control policy as a goal-conditional $\pi_\phi(\mathbf{a}_t|\mathbf{s}_t, \mathbf{g}_t) : \mathbb{G} \times \mathbb{S} \rightarrow \mathbb{A}$, where $\mathbf{g}_t \triangleq (\mathbf{m}_t, \mathbf{z}_t) \in \mathbb{G}$ is the goal at the time t that indicates the target of upper-body motion clip and the latent code from Encoder $E_\phi(\mathbf{z}_t|\mathbf{M}_t)$ from dataset. $\mathbf{s}_t \triangleq \{\mathbf{q}_t, \dot{\mathbf{q}}_t, \boldsymbol{\theta}_t, \boldsymbol{\omega}_t, \mathbf{a}_{t-1}, \mathbf{g}_t\} \in \mathbb{S}$ is the current observation, where $\mathbf{q}_t \in \mathbb{R}^{27}$, $\dot{\mathbf{q}}_t \in \mathbb{R}^{27}$ are the position and velocity of whole body joints, $\boldsymbol{\omega}_t$ is the angular velocity of the base, $\mathbf{a}_{t-1} \in \mathbb{R}^{12}$ means the last action of lower-body joint. $\mathbf{a}_t \in \mathbb{A}$ is the action of lower-body joints. Both of upper-body motions and lower-body actions are actuated by a PD torque controller $\boldsymbol{\tau}_t = k_p(\mathbf{a}_t - \mathbf{q}_t) + k_d\dot{\mathbf{q}}_t$ for each joint. The reward design is shown in Table II. It should be noted that the rewards for regularization of actions are used to shape the

standing mode, and the input history will be encoded as a hybrid internal embedding[31], which improves the training efficiency and robustness of lower-body RL controller.

In order to reduce the exploration burden caused by upper-body motions and gain more stable training process, we introduce a training curriculum schedule that will change the amplitude of target motion clips[7]. For the PD controller of the joint position during the training, the target joint position is calculated by

$$\mathbf{q}_t^{upper} = \mathbf{q}_{default}^{upper} + \alpha_i(\mathbf{q}_{target}^{upper} - \mathbf{q}_{default}^{upper}) \quad (5)$$

where $\mathbf{q}_{default}^{upper}$ is the default joint position of upper-body. The $\alpha_i \in [0, 1]$ is the unique amplitude factors of motion i , which is changed during the training by the rules similar to the [7].

D. Time Optimization Policy

We design a reinforcement learning policy to optimize the timestamp between motion clips $\mathbf{m}_t, \mathbf{m}_{t+1} \dots \mathbf{m}_{t+N}$, considering the latent variable $\mathbf{z}_t$, the current observation $\mathbf{s}_t$ and the history observation $\mathbf{h}_t$. The structure of this net can be described as $\Delta t_t^{seq} = \pi_\theta(\mathbf{m}_t, \mathbf{z}_t, \mathbf{s}_t, \mathbf{h}_t)$, the θ is the learnable variable of TOP. The output of TOP ($\Delta t_t^{seq} = \Delta t_t, \dots, \Delta t_{t+N}$) will feed into RL policy combined with the motion pair $(\mathbf{m}_t, \mathbf{z}_t)$. Once the timestamp of motion has been set to $t + \Delta t_t$, it is necessary to change the origin motion $\mathbf{m}_t$ to the new motion $\mathbf{m}'_t = \text{linear interpolate}(\mathbf{m}_{t-1}, \mathbf{m}_{t+\Delta t_t})$ by linear interpolation. Because the motions of the dataset $\mathcal{M}$ satisfy the kinematic and dynamic constraints of the robot, after linear interpolation, $\mathbf{m}'_t$ will not violate the constraints.

It should be noted that because of the rapid motion in the past few frames, the robot may lose its balance at the current moment, which exhibits a certain degree of lag. In other words, when we change the timestamp of the current upper-body motion to slow down the motion, it may be reflected in future multi frames feedback. This means that the output of the past policy (Δt_{t-N}) will affect the current balance performance of robot. Therefore, we are seeking to optimize timestamps for a period of time in the future, and inspired by *action chunking*[32], the policy model becomes $\pi_\theta(\Delta t_t^{seq} | \mathbf{m}_t, \mathbf{z}_t, \mathbf{s}_t, \mathbf{h}_t)$ instead of $\pi_\theta(\Delta t_t | \mathbf{m}_t, \mathbf{z}_t, \mathbf{s}_t, \mathbf{h}_t)$, $N = 10$ is the horizon step, which was obtained based on our testing.

A simple implementation of predicting a horizon of future timestamps like action chunking will be sub-optimal: if we directly shift by one timestamp until the end of trajectory, a new timestamps trajectory is incorporated abruptly every N steps and can result in the sudden action to slow down. To improve the action smoothness and avoid jerky discrete switching of timestamps, we query the output of policy at every timestamp, and give every prediction with an exponential weighting scheme $w_i = \exp(-k * i)$, where w_0 represents the weight of the oldest action. Then we use the weighted average for the current predictions similar to [12].

Obviously, this kind of feedback lag brings an extra burden of policy exploration and it is more difficult to predict a short horizon of timestamp trajectory than a single timestamp. It is natural for us to think that the original timestamp of the motion clips can be set as the initial guess solution for the policy.

Therefore, we introduce the supervise learning combined with reinforcement learning to reduce the need for inefficient random exploration, which also help steer the agent toward high-quality policies instead of suboptimal solutions [33]. Our PPO-based supervised reinforcement learning framework incorporates the following loss components:

$$L_{sup} = - \sum_i \Delta t_i^{log} \pi_\theta(\Delta t_i^* | \mathbf{s}_i) \quad (6)$$

$$L_{RL} = -\mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (7)$$

$$L_{total} = \lambda_{sup}L_{sup} + \lambda_{RL}L_{RL} \quad (8)$$

Where L_{sup}, L_{RL} are the loss of supervise and reinforcement learning. $r_t(\theta)$ is the policy ratio, and A_t is the advantage function. We combine both losses with weighting factors $\lambda_{sup} = 0.1$ and $\lambda_{RL} = 0.5$ to balance supervised learning and reinforcement learning. At the beginning of training, L_{sup} is relatively large and plays a dominant role. After L_{sup} decreases, the loss of L_{RL} plays a dominant role. And the rewards of TOP is shown in Table I.

We can easily observe that the reward design of TOP and the design of RL Rewards have similarities, as they both want to guide that robots can execute motions accurately while maintaining balance as much as possible. So why not train jointly? Because we want to make the TOP policy independent of the controller, we can retrain the top policy when the controller changes. In this way, we can freely change and update the controller without considering the top policy, and the performance of TOP policy will not impact the controller itself. During our training, we find that training both policy jointly will not affect the results of RL policy, but need more iteration to converge and the worse performance of TOP policy, which means separate training will bring more stable training results.

TABLE I: REWARDS OF TOP

Term	Expression	Weight
Gravity projection	$\exp(-20\ \mathbf{p}\mathbf{g}_t^{xy}\)$	2.5
Balance penalty	$\exp(20\sqrt{(\mathbf{p}\mathbf{g}_t^x)^2 + (\mathbf{p}\mathbf{g}_t^y)^2}) - 1$	-1.0
Support constraint	$\lg(7 * (\mathbf{p}_{feet}^{center} - \mathbf{p}_{com}^{project}))$	-5
Encourage small Δt	$\sum_{i=0}^N \exp\left(-\frac{(\Delta t_i)^2}{2\sigma^2}\right), \sigma = 0.5$	5.0
Δt smooth	$\sum_{i=0}^{N-1} (\ \Delta t_{i+1} - \Delta t_i\)$	-0.1
Δt norm	$\ \sum_{i=0}^N \Delta t_i\ $	0.1

IV. EXPERIMENTS

A. Experimental Setup

We validate our approach on a full-size humanoid robot in both simulation and real-world. The robot is 1.65m tall, weighs 60kg, and has 41 DoFs, including two 7-DoF arms (6kg each) with a 3kg payload per arm. This combination of weight, precision, and manipulation capability makes balance during manipulation particularly challenging. The training dataset $\mathcal{M}$ is the large-scale GRAB human motion dataset [34]. For testing, we additionally introduce more complex motions comprising 16,000 clips, forming dataset $\mathcal{T}$. All these motions are retargeted to humanoid robot via motion retargeting methods[35].

VAE We use a latent dimension of $d_z = 64$ and a window size of $W = 30$ (0.6s in real time). The VAE consists of

four 1D convolutional layers with Layer Normalization, ReLU activations, and a final linear layer. At the bottleneck, the encoder output is doubled and sampled from a multivariate Gaussian [4]. We adopt a β -VAE objective with a KL weight of 0.002 and a cyclical schedule to prevent KL collapse, combined with cosine annealing with warm restarts of learning rate for stable convergence. See Table II for hyperparameter details.

RL Policy We apply domain randomization for robustness to object properties, similar to [8]. Base CoM position, motor delays, and torque noise are critical for addressing the sim-to-real gap. We also add upper-body position and velocity noise during training to improve the robustness of the lower-body balance controller.

TOP Our TOP network uses a three-layer MLP and an actor-critic framework for stable learning. Supervised learning provides an initial guess for Δt^{seq} to encourage smaller Δt_t values. If the robot falls, the supervision loss L_{sup} is omitted from L_{total} , allowing the robot to slow down motions to maintain success. In practice, we limit Δt to $[0.01s, 0.1s]$. We find that action chunking with a weighting scheme and the motion prior are both critical for producing smooth, precise motions. Results are shown in table III.

TABLE II: VAE PARAMETERS AND RL REWARDS

VAE Parameters		VAE Training	
Param.	Value	Param.	Value
$kl\ weight$	0.002	Batch size	512
W	30	Number of epochs	30 000
d_z	64	Learning rate	0.003
Param.	0.8M	KL-scheduler cycles/ratio	7 / 0.5
Term	Expression	Weight	
Base linear Velocity ^{xy}	$\exp(-4(\mathbf{v}_{xy}^2))$	3.0	
Base linear Velocity ^z	$\Sigma \mathbf{v}_z^2$	-0.8	
Base angular Velocity ^{xy}	$\Sigma \omega_{xy}^2$	-0.1	
Base orientation	$\Sigma \mathbf{g}_{xy}^2$	-1.5	
Base acceleration	$\exp(-3\ \mathbf{v}_t - \mathbf{v}_{t-1}\ _2)$	0.2	
Stand still	$\exp(-10\ \mathbf{q}_t^{leg} - \mathbf{q}^{leg,ref}\ _2)$	1.0	
Feet contact	$\mathbf{1}(F_{feet}^z \geq 5)$	0.5	
Feet slip	$\mathbf{1}(F_{feet}^z \geq 5) \times \sqrt{\ \mathbf{v}_t^{feet}\ _2}$	0.2	
Action rate	$\ \mathbf{a}_t - \mathbf{a}_{t-1}\ _2^2$	-0.2	
Action acceleration	$\ \mathbf{a}_t + \mathbf{a}_{t-2} - 2\mathbf{a}_{t-1}\ _2^2$	-0.2	
Torques	$\ \boldsymbol{\tau}_t\ _2^2$	-5e-6	
DoF velocity	$\ \dot{\mathbf{q}}_t\ _2^2$	-5e-4	
DoF acceleration	$\ \ddot{\mathbf{q}}_t\ _2^2$	-1e-7	

B. Evaluation of Motion Priors and Decoupling Policy

Generalization of Motion Priors. To evaluate the efficacy of the VAE model and the latent space, we use linear interpolating between the latent space trajectories corresponding to two different motions, one motion comes from dataset $\mathcal{M}$, the other is an unseen motion from dataset $\mathcal{T}$. As shown in Figure 3, the result of 50% linear interpolation between two different latent code, and we reconstruct it by Decoder $D_\theta(\mathbf{M}'_t|z_t)$. We also show the reconstructed results of motions on the website, which demonstrate that our latent space can capture the features in short-horizon motions and has the ability to represent and reconstruct unseen motions that are in proximity to those of similar motion windows within the dataset.



Fig. 3: **Latent Generalization.** Given two motion windows, one from the dataset $\mathcal{M}$ (row 1), the other from unseen dataset $\mathcal{T}$ (row 3), and the results of 50% linear interpolation show in (row 2).

Robustness of Decoupling policy. To thoroughly evaluate the performance of our lower-body balance policy without TOP, we test upper-body motions from dataset $\mathcal{M}$ and unseen dataset $\mathcal{T}$. It is worth noting that the amplitude and speed of the motions from dataset $\mathcal{T}$ are larger than those in dataset $\mathcal{M}$, which put higher demands on the robustness. As shown in Figure 4, our policy can maintain balance while tracking upper-body motions with high precision and balance RL policy demonstrates strong robustness. Although there are occasional failures on the unseen dataset $\mathcal{T}$, particularly in challenging cases where the motion is too fast or both arms are raised above the head resulting in significant momentum changes that destabilize the robot. By slowing down such difficult motions, our system achieves a success rate of over 80%, indicating good generalization to unseen scenarios. The quantitative data are available in Table III.

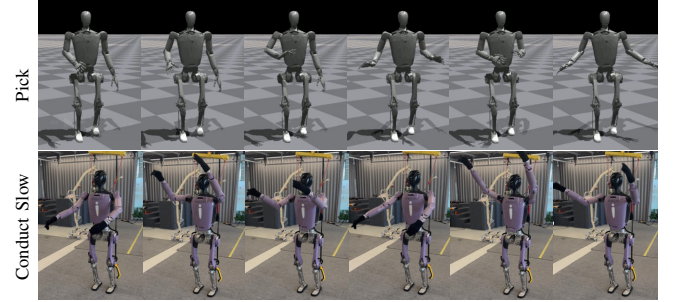


Fig. 4: **Robustness.** Given motion windows in simulation and real world, pick motions are sampled from the dataset $\mathcal{M}$ (row 1), and the conduct slower motions comes from unseen dataset $\mathcal{T}$ (row 2).

C. Evaluation of TOP.

- **Fixed Root (reference):** This baseline is that we fixed the root of robot, and directly execute the upper-body motions, which means the motions can be perfectly executed with the highest control accuracy.
- **Exbody[19]:** This baseline uses whole-body RL policy with the tracking rewards to control whole-body joints.
- **OmniH2O[18]:** This baseline uses whole-body RL policy with the tracking rewards to control whole-body joints, and similar to the Exbody in terms of reward settings and RL training.

Method	Time Cost ↓	Success Rate ↑	$E_{jpe}^{upper} ↓$	$E_{eepe}^{upper} ↓$	$E_{eeoe}^{upper} ↓$	$E_g ↓$	$E_{acc}^{lower} ↓$	$E_{action}^{lower} ↓$	$E_{acc}^{upper} ↓$	$E_{action}^{upper} ↓$
Comparative Results										
Fixed Root (reference)	15.0s	100.0%	0.0130	0.0164	0.0594	1.000	-	-	-	-
Exbody[19]	15.0s	92.46%	0.0376	0.0741	0.0923	3.432	21.65	0.7323	24.26	2.976
OmniH2O[18]	15.0s	94.08%	0.0361	0.0506	0.0914	3.267	19.86	0.6972	18.49	1.680
Mobile-Television[7]	15.0s	85.79%	0.0354	0.0513	0.0923	3.831	15.43	0.8775	11.67	1.677
NMPC+WBC[36]	35.0s	89.60%	0.0278	0.0437	0.0857	2.938	14.98	-	11.47	-
Ours (TOP)	40.5s	95.30%	0.0269	0.0270	0.0827	2.729	13.45	0.8592	10.41	1.601
Ablation Results										
Ours w/o TOP $\Delta t = 0.01s$	15.0s	82.43%	0.0301	0.0541	0.0962	4.438	16.87	0.9329	12.04	1.738
Ours w/o TOP $\Delta t = 0.03s$	45.0s	87.27%	0.0303	0.0434	0.0810	3.797	16.52	0.9473	11.26	1.647
Ours w/o TOP $\Delta t = 0.05s$	75.0s	92.41%	0.0299	0.0362	0.0813	3.573	15.79	0.9340	9.79	1.556
Ours w/o motion prior	34.5s	89.33%	0.0304	0.0313	0.0824	3.682	14.76	0.9130	10.92	1.640
Ours w/o acting chunking	28.5s	93.16%	0.0288	0.0353	0.0817	3.275	14.28	0.8835	11.51	1.659

TABLE III: Comparative results and Ablation results. We execute more than 10000 motion clips from dataset $\mathcal{M}$ and unseen dataset $\mathcal{T}$, and report their time cost, success rate, mean metrics. Mean Metrics are only calculated based on the data of the robot standing in place, and the data of robot taking steps and falls will not be included in the calculation.

- **Mobile-Television[7]**: This baseline uses decoupling controller with motion prior, which is similar to the **Ours w/o TOP** $\Delta t = 0.01s$. The code is not open source, so we implemented it on our own robot based on the reference paper.
- **NMPC+WBC[36]**: This baseline uses a non-linear Model Predictive Control (NMPC) combined with Whole-Body Control (WBC). And we slow down the target motions of NMPC for better comparison with our TOP method.
- **Ours (TOP)**: Our method of time optimization policy with balance lower-body RL policy.
- **Ours w/o TOP** $\Delta t = 0.01s/0.03s/0.05s$: These baselines only use the balance lower-body RL policy without TOP. The timestamp between motion clips is fixed as Δt .
- **Ours w/o motion priors**: This baseline uses TOP method but without the motion priors. The VAE module is entirely removed and the RL and TOP policies only based on the current motion state without any latent code.
- **Ours w/o acting chunking**: Our method of TOP, but the time optimization policy only predicts single Δt_t not a sequence of Δt_t^{seq} .

The metrics are as follows:

- **Time Cost**: Average time cost of per 1500 motion clips (15s of origin data), which shows the time efficiency of executing the target motions.
- **Success Rate**: We define that the robot executes the upper body motion without falling, this motion clip will be labeled as successful motion.
- **Precision**: upper joint position error E_{jpe}^{upper} , upper end effector position error in world frame E_{eepe}^{upper} , upper end effort orientation error in world frame E_{eeoe}^{upper} .
- **Stability**: projected gravity E_g , lower joint acceleration E_{acc}^{lower} , lower action difference E_{action}^{lower} .
- **Smoothness**: upper joint acceleration E_{acc}^{upper} , upper action difference E_{action}^{upper} .

Analysis of TOP methods. The results are shown in Table III. It is evident that, in comparison to the Exbody or OmniH2O method, the proposed method enhances the tracking accuracy of the joint position and end effector position, but has worse

time efficiency. We would like to clarify, this because our method will execute the upper-body motion directly, while the Exbody or OmniH2O method involves executing upper-body motions through an RL policy, which changes the amplitude of the robot’s upper-body motions to maintain balance. That is why these method performs worse in precision. And our method will consider the impact of momentum changes that will slow down the motion, leading to more time cost.

Considering the ablation results, as shown in the Figure 7, our method helps reduce the balance burden and unpredicted inference caused by the lower-body, which enhances both the balance stability and the precision of upper-body motions. And with the help of TOP algorithm, we have achieved a good balance between time efficiency, and accuracy, demonstrating the outperformance of our methods. Compare to **Ours w/o TOP** $\Delta t = 0.01s/0.03s/0.05s$, our method can automatically choose the appropriate $\Delta t \in [0.01s, 0.1s]$ in different situations: when motions seriously affect accuracy and balance, our algorithm will give a larger Δt even more than 0.05s to improve stability and precision; conversely, when motions are easy enough, a smaller Δt is chosen to maximize time efficiency. Although our methods take more time to complete motions compared to **Ours w/o TOP** $\Delta t = 0.01s$ and **Mobile-Television**, but we improve both precision and stability significantly. This adaptive TOP strategy results in improved overall precision and stability, while achieving higher average time efficiency. The performance curves of different methods over time are presented in Fig. 5, demonstrating the precision, stability and time efficiency of motion clips.

We also measure the real-world base and end effector poses within accurate motion capture device, which proves that our method can still maintain balance and demonstrate high accuracy considering the time efficiency of motions. Together with the robust lower-body controller, the adaptive TOP module plays a complementary role, and both are essential to achieve the observed improvements in overall performance. The quantitative results are shown in Table IV and Figure 6.

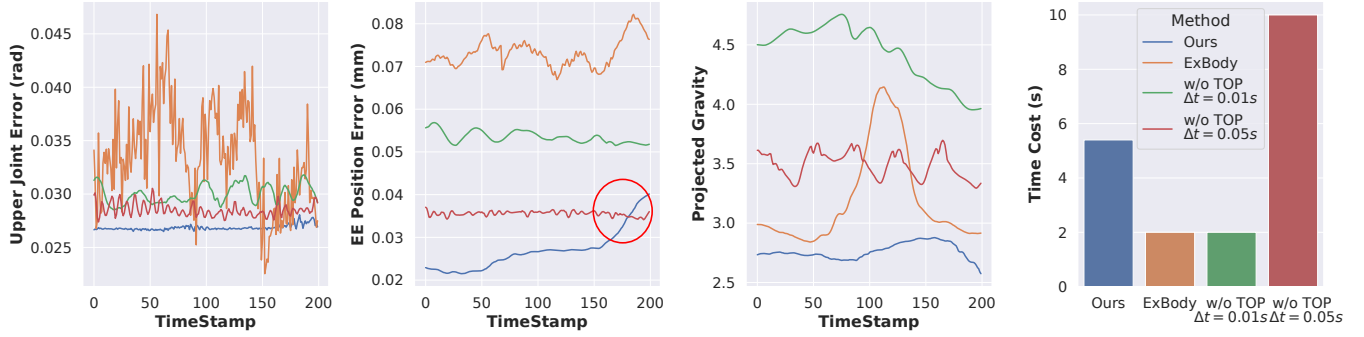


Fig. 5: Evaluation of precision, stability, and time efficiency. We only plot results for Exbody and w/o TOP ($\Delta t = 0.01s$), since Exbody and OmniH2O show very similar behavior, while Mobile-Television is comparable to w/o TOP ($\Delta t = 0.01s$). Red circle: Because upper-body motions can occasionally have a minor impact on whole-body balance, our method prefers to execute the motion at its original speed rather than slow it down. This may slightly increase the EE Position Error but represents a dynamic balance between precision, stability, and time efficiency.

TABLE IV: Real-world performance of several motions.

Part		x (mm)	y (mm)	z (mm)	Roll (rad)	Pitch (rad)	Yaw (rad)
$\mathcal{M}$	Base Error	1.654	0.461	0.123	0.0091	0.0114	0.0045
	EE Error	5.068	2.192	0.560	0.0213	0.0138	0.0048
$\mathcal{T}$	Base Error	29.232	2.485	1.169	0.0163	0.0119	0.0032
	EE Error	33.735	3.049	1.491	0.0502	0.0127	0.0103

¹ The errors are averaged over twenty different motions.

² Motions from $\mathcal{M}$ have smaller amplitude and slower speed of the motions than motions from $\mathcal{T}$.

³ It is evident that the error of the base accounts for the majority of the end-effector nose error.

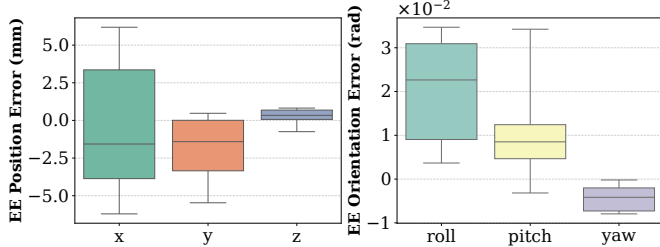


Fig. 6: **Evaluate of accuracy.** EE position and orientation error of our method. The motions are from $\mathcal{M}$ which includes the motion space and speed of the vast majority of manipulation tasks, and can well measure the accuracy of our algorithm in manipulation tasks.

D. Manipulation Experiments

We evaluate our approach on manipulation tasks that require precision and robustness simultaneously, and we also use teleoperation combined with TOP for real-time motion slowing. In simple manipulation tasks, like grab a cup, policy tends to maintain the original speed to get higher time efficiency. When the amplitude and speed of the robot’s motion affect balance, like dance with arms, it will actively slow down the motion, which can reduce the impact of momentum on the robot to control the robot stably and accurately. And we also performs loco-manipulation tasks, the results are shown in Figure 8.

V. CONCLUSIONS

In this paper, we propose a novel framework with the TOP method, which improves lower-body stability and upper-body precision by optimizing the timestamps of motion clips. By integrating a robust lower-body policy with precise upper-body tracking, and leveraging a VAE-based motion prior to capture

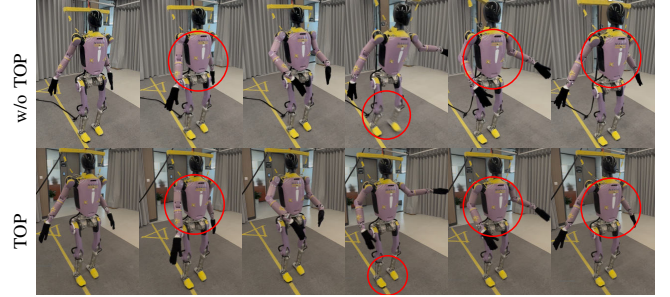


Fig. 7: **TOP Performance.** The result shows that TOP can improve the stability and reduce the occurrence of robot taking a step or falling in real world.

upper-body features, our method achieves 95.30% success rates in experiments, as reported in table III, demonstrating stable and accurate standing manipulation performance. However, current motions remain somewhat rigid, for instance, the robot tends to tilt backward rather than twisting its hips. In future work, we plan to incorporate motion generation modules to enable more natural standing adjustments and further enhance balance control. Moreover, we aim to explore adaptive task-oriented time optimization policy to extend our application in real-world tasks.

REFERENCES

- [1] Zhaoyuan Gu et al. “Humanoid locomotion and manipulation: Current progress and challenges in control, planning, and learning”. In: *arXiv preprint arXiv:2501.02116* (2025).
- [2] Qingwei Ben et al. “HOMIE: Humanoid Loco-Manipulation with Isomorphic Exoskeleton Cockpit”. In: *arXiv preprint arXiv:2502.13013* (2025).
- [3] Giulio Romualdi et al. “Online non-linear centroidal mpc for humanoid robot locomotion with step adjustment”. In: *2022 International Conference on Robotics and Automation (ICRA)*. IEEE. 2022, pp. 10412–10419.
- [4] Agon Serifi et al. “Vmp: Versatile motion priors for robustly tracking motion on physical characters”. In: *Computer Graphics Forum*. Vol. 43. 8. Wiley Online Library. 2024, e15175.
- [5] Mazeyu Ji et al. “Exbody2: Advanced expressive humanoid whole-body control”. In: *arXiv preprint arXiv:2412.13196* (2024).
- [6] Mingyo Seo et al. “Deep imitation learning for humanoid loco-manipulation through human teleoperation”. In: *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*. IEEE. 2023, pp. 1–8.



Fig. 8: **Manipulation tasks.** (a) grab the cup and put it onto cups, (b) take and deliver a bottle of water, (c) dance with arms but slow down the original motions while the lower body maintains balance, (d) open the oven, (e) Take the book and pass it from left hand to right hand, (f) carry a box. (g) Loco-open-fridge, (h) loco-button-pressing task.

- [7] Chenhao Lu et al. “Mobile-television: Predictive motion priors for humanoid whole-body control”. In: *arXiv preprint arXiv:2412.07773* (2024).
- [8] Ilija Radosavovic et al. “Real-world humanoid locomotion with reinforcement learning”. In: *Science Robotics* 9.89 (2024), eadi9579.
- [9] Zipeng Fu et al. “HumanPlus: Humanoid Shadowing and Imitation from Humans”. In: *arXiv preprint arXiv:2406.10454* (2024).
- [10] Minghuan Liu et al. “Visual whole-body control for legged loco-manipulation”. In: *arXiv preprint arXiv:2403.16967* (2024).
- [11] Songming Liu et al. “Rdt-1b: a diffusion foundation model for bimanual manipulation”. In: *arXiv preprint arXiv:2410.07864* (2024).
- [12] Tony Z Zhao et al. “Learning fine-grained bimanual manipulation with low-cost hardware”. In: *arXiv preprint arXiv:2304.13705* (2023).
- [13] Yasuhiro Ishiguro et al. “Bipedal oriented whole body master-slave system for dynamic secured locomotion with lip safety constraints”. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2017, pp. 376–382.
- [14] Luigi Penco et al. “A multimode teleoperation framework for humanoid loco-manipulation: An application for the icub robot”. In: *IEEE Robotics & Automation Magazine* 26.4 (2019), pp. 73–82.
- [15] Zhengyi Luo et al. “Perpetual humanoid control for real-time simulated avatars”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 10895–10904.
- [16] Nicklas Hansen et al. “Hierarchical World Models as Visual Whole-Body Humanoid Controllers”. In: *arXiv preprint arXiv:2405.18418* (2024).
- [17] Qiang Zhang et al. “Whole-body humanoid robot locomotion with human reference”. In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2024, pp. 11225–11231.
- [18] Tairan He et al. “OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning”. In: *arXiv preprint arXiv:2406.08858* (2024).
- [19] Xuxin Cheng et al. “Expressive whole-body control for humanoid robots”. In: *arXiv preprint arXiv:2402.16796* (2024).
- [20] Sebastian Starke, Ian Mason, and Taku Komura. “Deepphase: Periodic autoencoders for learning motion phase manifolds”. In: *ACM Transactions on Graphics (TOG)* 41.4 (2022), pp. 1–13.
- [21] Xue Bin Peng et al. “Ase: Large-scale reusable adversarial skill embeddings for physically simulated characters”. In: *ACM Transactions On Graphics (TOG)* 41.4 (2022), pp. 1–17.
- [22] Xue Bin Peng et al. “Amp: Adversarial motion priors for stylized physics-based character control”. In: *ACM Transactions on Graphics (ToG)* 40.4 (2021), pp. 1–20.
- [23] Mohamed Hassan et al. “Synthesizing physical character-scene interactions”. In: *ACM SIGGRAPH 2023 Conference Proceedings*. 2023, pp. 1–9.
- [24] Annan Tang et al. “Humanmimic: Learning natural locomotion and transitions for humanoid robot via wasserstein adversarial imitation”. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2024, pp. 13107–13114.
- [25] Félix G Harvey et al. “Robust motion in-betweening”. In: *ACM Transactions on Graphics (TOG)* 39.4 (2020), pp. 60–1.
- [26] Wenyang Zhou et al. “Emdm: Efficient motion diffusion model for fast and high-quality motion generation”. In: *European Conference on Computer Vision*. Springer. 2024, pp. 18–38.
- [27] Jungdam Won, Deepak Gopinath, and Jessica Hodgins. “Physics-based character controllers using conditional vaes”. In: *ACM Transactions on Graphics (TOG)* 41.4 (2022), pp. 1–12.
- [28] Guy Tevet et al. “Human motion diffusion model”. In: *arXiv preprint arXiv:2209.14916* (2022).
- [29] Jiefeng Li et al. “GENMO: A GENeralist Model for Human MOTion”. In: *arXiv preprint arXiv:2505.01425* (2025).
- [30] Irina Higgins et al. “beta-vae: Learning basic visual concepts with a constrained variational framework.” In: *ICLR (Poster)* 3 (2017).
- [31] Junfeng Long et al. “Hybrid internal model: Learning agile legged locomotion with simulated robot response”. In: *arXiv preprint arXiv:2312.11460* (2023).
- [32] Lucy Lai, Ann Zixiang Huang, and Samuel J Gershman. “Action chunking as policy compression”. In: (2022).
- [33] Lu Wang et al. “Supervised reinforcement learning with recurrent neural network for dynamic treatment recommendation”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2018, pp. 2447–2456.
- [34] Omid Taheri et al. “GRAB: A dataset of whole-body human grasping of objects”. In: *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part IV 16*. Springer. 2020, pp. 581–600.
- [35] Haodong Zhang et al. “Kinematic motion retargeting via neural latent optimization for learning sign language”. In: *IEEE Robotics and Automation Letters* 7.2 (2022), pp. 4582–4589.
- [36] Ruben Grandia et al. “Perceptive locomotion through nonlinear model-predictive control”. In: *IEEE Transactions on Robotics* 39.5 (2023), pp. 3402–3421.
- [37] Yusuf Aydin et al. “A computational multicriteria optimization approach to controller design for physical human-robot interaction”. In: *IEEE Transactions on Robotics* 36.6 (2020), pp. 1791–1804.
- [38] Tilen Breclj and Tadej Petrič. “Zero moment line—universal stability parameter for multi-contact systems in three dimensions”. In: *Sensors* 22.15 (2022), p. 5656.

VI. APPENDIX

A. Motion Alignment And Coupling Effects

In principle, upper- and lower-body motions are inherently coupled through momentum transfer and base motion feedback. Fully ignoring this coupling would risk infeasible or unstable motions. In our design, however, the decoupling is a practical modular abstraction rather than a rigid separation. Upper-body motion is compressed by a VAE-based motion prior and tracked by a PD controller that outputs desired joint torques to follow the reference trajectories with high precision, while lower-body RL policy receives real-time observations of the robot state, and the desired upper-body motion embedding (z_t) and motion clip (m_t), compensating for the resulting momentum changes by dynamically regulating base drift and joint torques.

Therefore, while the modules are trained in a decoupled manner, their execution forms a closed-loop whole-body control system. The RL policy implicitly compensates for momentum transfer from the upper-body by dynamically adjusting lower-body joint torques and contact forces to maintain balance. This design principle is consistent with established whole-body control frameworks[7] where manipulation and locomotion are hierarchically coordinated. Our reward structure explicitly penalizes excessive base drift and velocity deviations, enforcing feasible whole-body dynamics, as shown in the table II.

As shown in table III, compared to baseline methods (Exbody[19], OmniH2O[18]) that rely on end-to-end whole-body RL tracking, our hybrid modular strategy achieves higher tracking precision, base stability, and overall motion success rate. One detail worth clarifying is the slightly higher upper-body acceleration error observed for our method (TOP) relative to the baseline (**Ours w/o TOP** $\Delta t = 0.05s$). This effect stems from our time optimization policy, which adaptively adjusts the execution speed of the upper-body trajectory in response to the robot's real-time dynamic state. Such non-uniform time scaling naturally introduces local variations in velocity and acceleration, resulting in occasional mild phase lags or higher acceleration spikes when the speed changes rapidly. In contrast, the baseline uses a fixed time step for the entire trajectory, which reduces local acceleration fluctuations but sacrifices dynamic adaptability and time efficiency.

In summary, although the TOP policy dynamically adjusts upper-body timestamps, the lower-body actions are not static but adapt in real-time to ensure whole-body consistency. The empirical results and reward design confirm that our approach avoids dynamic misalignment and preserves stable and feasible behavior.

B. Quantitative Regression Analysis

fig. 5 qualitatively illustrated the great balance between precision, stability, and time efficiency. To provide a more rigorous quantitative characterization of these interdependencies, we additionally conducted a regression and Pareto analysis.

Specifically:

- Multiple linear and polynomial regressions were applied to model the relationship between time efficiency (Δt),

base stability (projected gravity), and precision (end-effector position error).

- An empirical Pareto front[37] was computed to reveal the boundary of non-dominated solutions, illustrating the inherent trade-offs.

fig. 9 shows the updated results. Each scatter point represents an individual upper-body motion trial with its measured Δt , projected gravity, and pose error. The fitted regression surface captures the trend that smaller Δt generally leads to slightly higher projected gravity and pose error due to increased momentum transfer from rapid upper-body motions. The highlighted Pareto front marks the boundary where improvement in one metric cannot be achieved without compromising at least one of the others.

This analysis confirms that higher end-effector accuracy is not independent but is coupled to base stability and Δt . Larger Δt implies slower motions, allowing the lower-body controller more time to absorb upper-body momentum, resulting in lower projected gravity and reduced pose error. Conversely, executing motions faster increases local accelerations, slightly degrading base stability and precision.

C. Robustness Evaluation

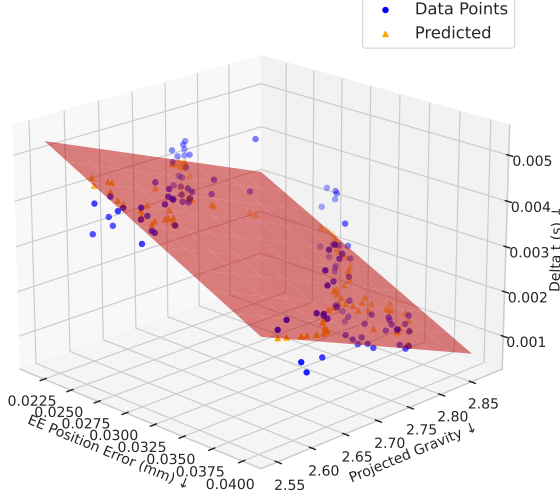
TABLE V: Domain Randomization Settings

Term	Value
Observation	
Joint position noise	$\mathcal{U}(-0.1, 0.1)$ rad
Joint velocity noise	$\mathcal{U}(-1.0, 1.0)$ rad/s
Angular velocity noise	$\mathcal{U}(-0.1, 0.1)$ rad/s
Projected gravity noise	$\mathcal{U}(-0.05, 0.05)$
Dynamics	
Base COM displacement	$\mathcal{U}(-0.05, 0.05)$ m
Friction factor	$\mathcal{U}(0.35, 1.25)$
Link Inertia	$\mathcal{U}(0.8, 1.2) \times \text{default kg} \cdot \text{m}^2$
Added base mass	$\mathcal{U}(-5.0, 5.0) + \text{default kg}$
Added link mass	$\mathcal{U}(0.8, 1.2) \times \text{default kg}$
Added hand mass	$\mathcal{U}(-0.5, 1.5) + \text{default kg}$
Payload mass	$\mathcal{U}(-1.0, 3.5) + \text{default kg}$
Motor P gain	$\mathcal{U}(0.8, 1.2) \times \text{default}$
Motor D gain	$\mathcal{U}(0.8, 1.2) \times \text{default}$
Motor strength	$\mathcal{U}(0.9, 1.1) \times \text{default N} \cdot \text{m}$
Motor Damping	$\mathcal{U}(0.3, 4.0)$ N · s
Motor Delay	$\mathcal{U}(0, 10)$ ms
Episode	
Push robots	interval = 5s, $v_{xy} = 1.0\text{m/s}$, $\omega = 0.4\text{rad/s}$
Disturbance robots	interval = 8s, $\mathcal{U}(-30.0, 30.0)$ N · m
Initial robot base position xy	$\mathcal{U}(-1.0, 1.0)$ m
Initial robot base velocity	$\mathcal{U}(-0.1, 0.1)$ m/s

Although our VAE focuses on learning compact latent representations of human-like motion purely from joint position and velocity data, without directly incorporating dynamic constraints (e.g., torque limits, inertia matrices). However, the final execution relies on a downstream reinforcement learning (RL) policy, and we incorporate extensive domain randomization during RL training. As summarized in table V, this includes random variations in payload mass, link inertia, base mass, motor properties, friction, and observation noise. In addition, randomized external pushes and initial state perturbations are applied to expose the policy to diverse disturbance scenarios.

To empirically verify this design, we conducted an additional load-adaptability test, in which the robot performed the same manipulation task while grasping objects of increasing

Regression Plane: Time Efficiency vs. Precision & Stability



3D Convex Hull (Approximate Pareto Front)

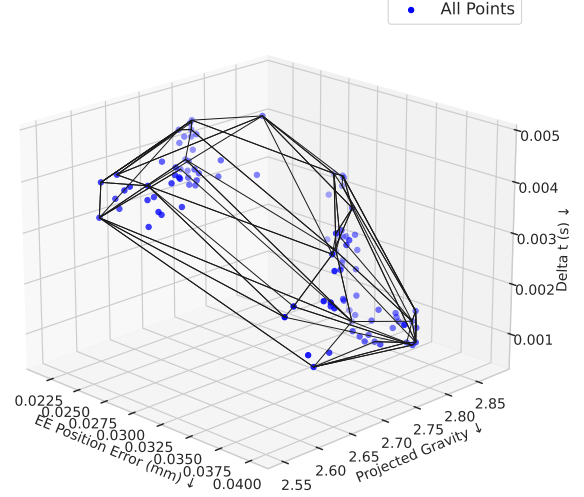


Fig. 9: **Regression and Pareto front analysis.** The figure shows the multi-objective trade-off among time, stability, and precision, with the fitted regression surface and the extracted Pareto optimal front (right).

mass. table VI reports the results. The task success rate and end-effector pose error remain stable across different payloads, while only a slight increase in the projected gravity term is observed. This demonstrates that our RL controller compensates for the dynamic variations introduced by heavier payloads, ensuring that the generated actions remain physically feasible and stable even though the upstream VAE module is purely kinematic.

TABLE VI: Payloads Adaptability Test

Payload	Success Rate $\uparrow$	EE Pose Error $\downarrow$	Projected Gravity $\downarrow$
0.5 kg	95.2%	28.5 mm	2.731
1.0 kg	94.4%	30.1 mm	2.857
3.0 kg	92.7%	31.8 mm	3.142

Also, we add more experiments about push recovery of our method in real-world robot. fig. 10 shows external disturbance tests of our policy. We designed scenarios where the robot is intentionally disturbed by external pushes from different directions.

Moreover, we have conducted additional experiments that extend beyond static standing manipulation. We now demonstrate the effectiveness and robustness of our Time Optimization Policy (TOP) when the robot performs loco-manipulation tasks. fig. 11 shows the mobile operation scenarios of our TOP policy. In these new test cases, the humanoid robot executes upper-body tasks and omnidirectional walking. This demonstrates that the proposed decoupled yet closed-loop framework generalizes well to locomotion plus manipulation tasks.

To make our results fully transparent, we have released all the demonstration videos on our project page <https://anonymous.4open.science/w/top-258F/>.

D. Measurement of Manipulation Tasks

To complement the qualitative demonstrations in fig. 8, we report detailed quantitative performance metrics for each manipulation scenario. fig. 12 presents the **success rate**, **task completion time**, and **end-effector pose error** (EE pose error)

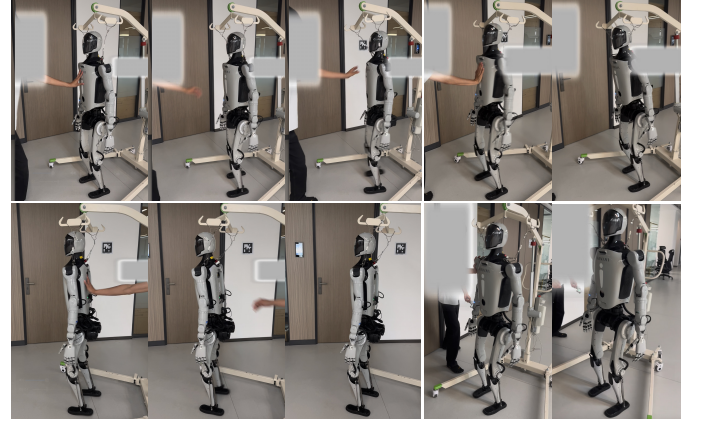


Fig. 10: **External push disturbance tests.** The figure illustrates the robot can recover to balance while disturbed by external pushes from different directions.

for five representative tasks: grabbing a cup, taking water, opening an oven, taking a book, and carrying a box. These metrics were obtained by repeating the real-world experiments multiple times under the same conditions.

The results demonstrate that our framework achieves high success rates and acceptable completion times while maintaining reasonable EE pose errors. For more challenging tasks such as *take book* and *carry box*, slightly higher pose errors are observed due to the increased complexity of manual operation and shifting loads, which introduce additional vibrations and tracking challenges on the real robot. Despite these challenges, the success rates remain competitive (around 85%), showcasing the robustness of our method in realistic settings.

To provide more fine-grained, task-specific precision metrics as requested by the reviewer, we additionally measured detailed indicators such as handle rotation angle error and final placement offsets using a millimeter ruler, due to the difficulty of motion capture device handling the reflective objects. These results are summarized in table VII. For example, the oven handle rotation error is kept below 5° , and object



Fig. 11: **Loco-manipulation scenarios.** The top figures show the robot executing a loco-button-pressing task. The bottom figures shows the robot executing a loco-open-fridge task.

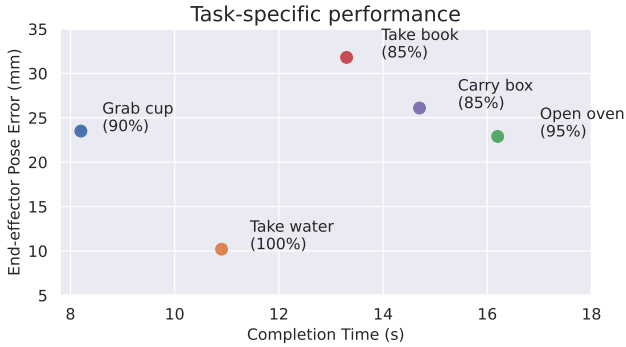


Fig. 12: **Comparison between different manipulation tasks.** Where each point represents a single task annotated by its success rate. The closer the data points are to the lower left corner, the lower the end-effector pose error and the shorter the task completion time, indicating better time efficiency and precision. This trend demonstrates how our method maintains high success rates while balancing precision and time efficiency across tasks of varying complexity.

placement offsets remain within a few centimeters, verifying that the system achieves both reliable general pose tracking and practical task-level accuracy.

TABLE VII: Detailed Task-Specific Metrics

Task	Key Precision Metric	Value
Grab cup	Cup final placement offset (mm)	22.5 mm
Open oven	Handle rotation angle error (deg)	4.8°
Take book	Book alignment deviation (mm)	28.7 mm
Carry box	Box final placement offset (mm)	25.3 mm

Together, these results provide solid evidence that our method achieves reliable and efficient performance across diverse manipulation tasks, validating its practicality under real-world uncertainties.

E. Inference Performance and Hardware Details

To quantify the computational overhead of our system, we measured the inference performance of each core module, including the VAE encoder, the RL policy, and the TOP policy. table VIII summarizes the average latency, device

specifications, and resource utilization for both high-end GPU (RTX 4090) and edge computing (PICO-TGU4) platforms.

TABLE VIII: Inference Performance in Difference Devices.

Module	Latency (ms)	Device	Resource Utilization
VAE Encoder	1.0121	RTX 4090 GPU	GPU Utilization: 4%
RL policy	1.7414	RTX 4090 GPU	GPU Utilization: 5%
TOP Policy	1.8623	RTX 4090 GPU	GPU Utilization: 8%
Total Inference	4.6158	RTX 4090 GPU	Memory Usage: 4.410 GB
VAE Encoder	1.4621	PICO-TGU4	CPU Utilization: 6%
RL policy	1.6925	PICO-TGU4	CPU Utilization: 7%
TOP Policy	1.7267	PICO-TGU4	CPU Utilization: 7%
Total Inference	4.8813	PICO-TGU4	Memory Usage: 5.4339 GB

In actual deployment, all trained .pt models are exported to the ONNX format for optimized runtime inference. Since the network architectures are lightweight and do not contain complex structures such as Transformers, the inference is highly efficient and stable across platforms.

Overall, the combined inference time is approximately **5 ms**, enabling a control frequency of 100 Hz, which demonstrates that the proposed system is suitable for real-time applications.

In addition, to provide a clearer understanding of the experimental setup of our system, we report the detailed hardware configuration below. The humanoid robot used in our simulations and experiments stands 1.65 meters tall, weighs 60 kg, and has 41 degrees of freedom, including two 7-DoF arms (6 kg each) with a payload capacity of 3 kg per arm. fig. 13 summarizes the robot's drive hardware and onboard computing unit configuration.

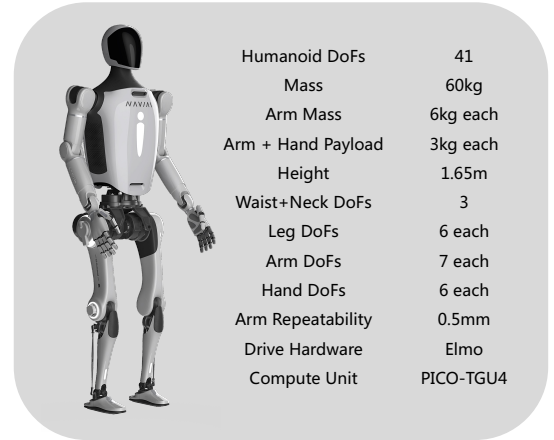


Fig. 13: **Hardware Details.** Our robot stands 1.65m tall, weighs 60kg, and has 41 degrees of freedom, including two 6kg arms with 7-DoF each and a payload capacity of 3kg per arm.

These details ensure transparency and demonstrate that our proposed system is feasible for real-time deployment on practical hardware.

F. Sensitivity Analysis of the impact of k

To improve the Δt smoothness of the predicted timestamps and avoid jerky discrete switching, our method uses a decay coefficient k in the action chunking[12] mechanism. The decay coefficient k defines a weight vector for aggregating recent actions when computing the final timestamp adjustment Δt .

Specifically, for a buffer of N past frames, the weight for frame i is given by:

$$w_i = \frac{\exp(-k \cdot i)}{\sum_{j=0}^N \exp(-k \cdot j)}, \quad i = 0, 1, \dots, N. \quad (9)$$

This assigns higher weights to recent actions and exponentially down-weights older ones. A smaller k yields a flatter weight distribution (more uniform smoothing), while a larger k makes the system more reactive to the latest changes. The chunking mechanism thus balances temporal consistency and reactivity by tuning k .

We conducted a sensitivity study by varying k in 0.0, 0.2, 0.5, 0.8, 1.5, 3.0 and measuring its impact on joint tracking error, base stability (projected gravity), execution time cost, and success rate. fig. 14 summarize the results.

When k is very small ($k = 0.0, 0.2$), the weights are nearly uniform, which overly smooths the timestamps and reduces responsiveness to rapid state changes. When k is too large ($k = 1.5, 3.0$), the chunking emphasizes only the most recent action, losing the benefit of temporal consistency and slightly increasing error and base drift. A moderate value ($k = 0.5$) achieves the best balance: it yields the lowest joint error (0.0269 rad) and the lowest projected gravity (2.7290), with a reasonable time cost and the highest task success rate (95.3%). This empirical result supports the choice of $k = 0.5$ as the default setting for our time optimization policy.

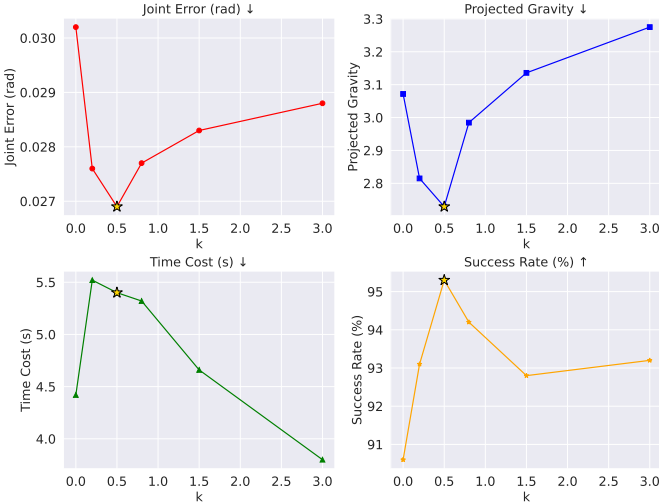


Fig. 14: **Sensitivity Analysis of Decay Coefficient k .** The plots show how k affects (a) Joint Error, (b) Projected Gravity, (c) Time Cost, and (d) Success Rate. The golden point marks $k = 0.5$, which achieves the best overall trade-off.

G. Sensitivity Analysis of VAE parameters

The latent code dimension and window size are critical hyperparameters for ensuring that the VAE can effectively extract meaningful motion priors. Therefore, we conducted an ablation study to evaluate how these settings affect the reconstruction quality, KL divergence loss, and overall generalization.

table IX summarizes the results. A moderate window size of 30 frames achieves the lowest reconstruction error and stable KL loss on both training and test sets. Shorter window size

(e.g. 15) provide insufficient temporal context for modeling complex motion patterns, which leads to poorer test performance despite similar training loss. Conversely, overly long windows size (e.g. 60) increase the temporal span to be jointly encoded, introducing noise, higher variance, and optimization difficulty, which degrades test quality due to overfitting.

TABLE IX: Sensitivity to Latent Dimension and Window Size

Window Size	Latent Dim	Recon. Loss Train	Recon. Loss Test	KLD Loss Train	KLD Loss Test	Time (hrs)
15	64	0.0261	0.3751	0.0148	0.0207	3.6
30	64	0.0250	0.2495	0.0126	0.0143	4.2
60	64	0.0714	0.6992	0.0081	0.0096	4.7
30	32	0.1139	2.7729	0.0086	7.8787	4.0
30	64	0.0250	0.2495	0.0126	0.0143	4.2
30	128	0.0238	0.2807	0.0118	0.0161	5.3

Regarding latent dimension, 64 achieves the best trade-off between representation capacity and regularization. A smaller code (32) lacks sufficient capacity, resulting in high test reconstruction and KL losses and unstable latent encoding. A larger dimension (128) slightly improves training performance but increases test error and computational cost due to overfitting and redundant capacity.

H. Analysis of Stability

To provide theoretical evidence of whole-body stability, we complement our empirical results with an explicit analysis based on the Zero Moment Point (ZMP) criterion. The ZMP is defined as the point on the ground plane where the horizontal component of the moment generated by the ground reaction force becomes zero. A necessary condition for dynamic equilibrium is that the ZMP must remain inside the support polygon formed by all contact points. To generalize this idea to non-planar contact scenarios, we adopt the concept of the Zero Moment Line (ZML) [38], which extends ZMP to varying contact heights.

The ZMP is calculated by solving:

$$\tau = p_{zmp} \times f + \tau_p \quad (10)$$

$$\dot{P} = Mg + f \quad (11)$$

$$\dot{L} = p_{com} \times Mg + \tau \quad (12)$$

where p_{zmp} is the position of ZMP, τ_p is the moment about the ZMP point, and the f represents the ground reaction force. P , L , M , p_{com} represents the linear momentum, the angular momentum, the total mass, the center of mass (CoM) position of the system, respectively, and $g = [0, 0, -g]^T$. By the definition of ZMP, we have:

$$\tau_{p,x} = \tau_{p,y} = 0 \quad (13)$$

Solving the above equations with respect to p_{zmp} , we have

$$p_{zmp,x} = \frac{Mgp_{com,x} + p_{com,z}\dot{P}_x - \dot{L}_y}{Mg + \dot{P}_z} \quad (14)$$

$$p_{zmp,y} = \frac{Mgp_{com,y} + p_{com,z}\dot{P}_y + \dot{L}_x}{Mg + \dot{P}_z} \quad (15)$$

By varying the height $p_{com,z}$, these equations generate the ZML, whose projection must intersect the support polygon to ensure balance, as illustrated in fig. 15.

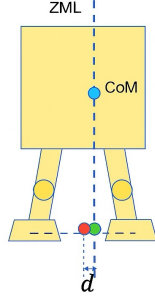


Fig. 15: **ZML cross the support polygon.** The red dot shows the approximate center of the support region; the green dot shows the projected ZML point. The distance d quantifies stability: smaller d implies better dynamic balance.

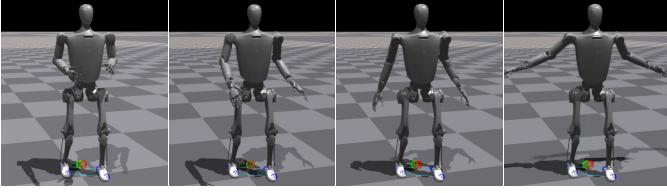


Fig. 16: **ZML and Support Polygon in Isaacgym.** The red point represents the center of the support polygon, and the green dot on the ZML is the projected point at the same horizontal height as the red dot. The blue points and the light blue lines are the support polygon.

In practice, the support polygon is computed as the convex hull of all current contact points. The ZML is computed in simulation for all time steps (fig. 16). We define the stability metric d as:

$$d = \min_{p_{csp} \in \mathcal{S}} \|p_{csp} - \text{proj}_{ZML}(p_{csp})\|_2 \quad (16)$$

where $\mathcal{S}$ denotes the support polygon of current contact points, and $\text{proj}_{ZML}(p_{csp})$ is the projected ZML point in the horizontal plane. If $d \in [0, 0.32]$, the ZML point is inside the support polygon; if $d \in (0.32, 0.36]$, it lies near the edge of support polygon; and if $d \in (0.36, +\infty)$, the ZML has exited the support region, indicating potential instability. It is worth noting that the closer d is to 0, the system is more stable and satisfy the ZMP constraint.

To verify that our TOP satisfies this condition, fig. 17 shows the time series of d for representative tasks. The results confirm that d consistently remains below 0.32, proving that the ZML stays well inside the support polygon during dynamic execution, even with rapid upper-body motions.

These results theoretically validate that our closed-loop controller robustly maintains dynamic stability by keeping the ZMP within feasible bounds, complementing the empirical results and reinforcing the soundness of our framework.

I. Analysis of Momentum

To rigorously analyze how fast upper-body motions can induce momentum disturbances and compromise whole-body stability, we provide both theoretical and empirical evidence.

First, the total angular momentum L of the robot can be expressed as the sum of the floating base angular momentum

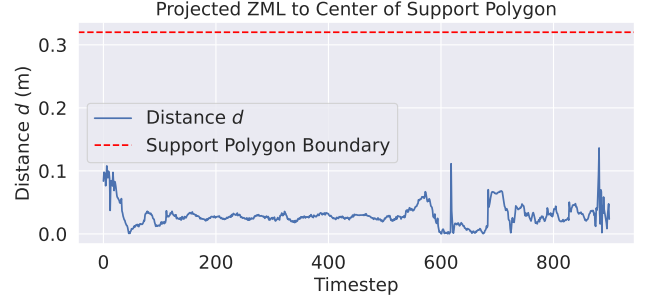


Fig. 17: **Stability metric d over time.** The curve shows the distance between the ZML projection and the center of support polygon. The distance remains ≤ 0.32 , confirming that the ZMP condition is always satisfied.

and the contribution from each upper-body joint:

$$L = I_B \omega_B + \sum_{i=1}^n I_i \dot{q}_i, \quad (17)$$

where I_B and ω_B denote the inertia tensor and angular velocity of the base, respectively, and I_i , $\dot{q}_i$ are the inertia and velocity of joint i . Taking the time derivative shows that rapid changes in joint accelerations $\ddot{q}_i$ directly induce compensatory reactions on the base:

$$I_B \dot{\omega}_B = - \sum_{i=1}^n I_i \ddot{q}_i. \quad (18)$$

This relationship indicates that large joint accelerations generate equal and opposite angular impulses on the floating base, which can destabilize the robot if left unregulated. Integrating over time confirms that the base's angular drift is proportional to the accumulated joint accelerations:

$$\Delta \omega_B \propto - \int_{t_0}^t \sum_{i=1}^n I_i \ddot{q}_i dt. \quad (19)$$

Moreover, faster joint velocities $\dot{q}_i$ also amplify the total angular momentum, further stressing the need for regulating motion speed. To address this, our Time Optimization Policy (TOP) explicitly shapes the motion timing to keep both $\dot{q}_i$ and $\ddot{q}_i$ within safe bounds, thereby suppressing sudden momentum spikes and preventing undesired base rotations.

To validate this mechanism in practice, fig. 18 shows the measured floating base velocity and acceleration during a fast upper-body motion. The results demonstrate that the TOP effectively smooths out abrupt base movements by adaptively slowing the arm when the momentum is excessive. This mitigation helps maintain the Zero Moment Point (ZMP) inside the support polygon, ensuring stable whole-body balance.

Overall, this momentum analysis clarifies why simply decoupling the upper-body motion without proper time optimization can lead to significant base disturbance, and highlights the necessity of regulating joint accelerations and velocities to maintain dynamic stability.

J. Training Efficiency

To provide a fair assessment of the training cost of our approach, we report the training time of each module and

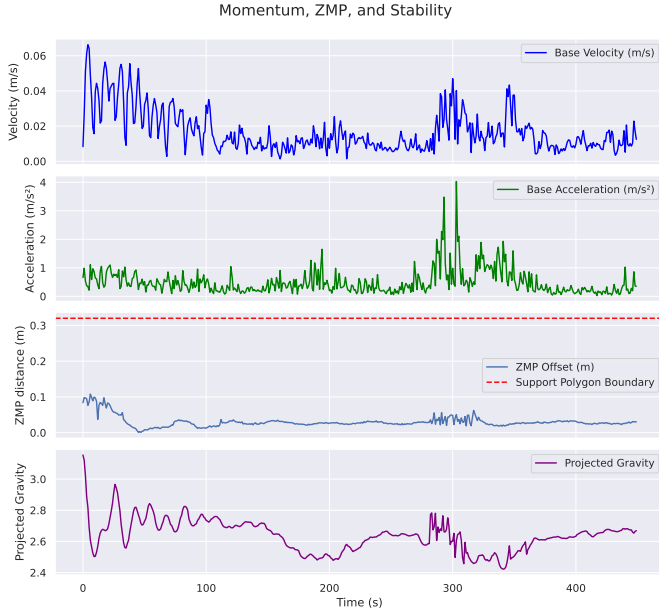


Fig. 18: **Momentum, ZMP and stability.** The figure shows the measured floating base velocity and acceleration during a fast upper-body motion. The TOP smooths out large momentum variations, helping to keep the ZMP within the support region and maintain balance.

compare the overall pipeline with relevant baselines such as Exbody, OmniH2O, and Mobile-Television.

Specifically, our framework requires approximately 10 hours to train the VAE motion prior, and about 5 hours each for the reinforcement learning (RL) policy and the time-optimized policy (TOP) module, when trained on a single NVIDIA RTX 4090 GPU. In total, our pipeline completes training in roughly 20 hours.

By contrast, training Exbody and OmniH2O under the same hardware setting takes around 22–32 hours, as these baselines require longer joint optimization of imitation and policy modules without task-specific time adaptation. As summarized in table X, our method achieves comparable or better training efficiency while providing adaptive timing and stable manipulation capabilities.

TABLE X: Comparison of Training Efficiency

Method	Hardware	Total Training Time (hrs)
VAE	1x RTX 4090 (24GB)	10 (approx.)
RL	1x RTX 4090 (24GB)	5 (approx.)
TOP	1x RTX 4090 (24GB)	5 (approx.)
Ours (VAE + RL + TOP)	1x RTX 4090 (24GB)	20 (approx.)
Exbody[19]	1x RTX 4090 (24GB)	25 (approx.)
OmniH2O[18]	1x RTX 4090 (24GB)	32 (approx.)
Mobile-Television[7]	1x RTX 4090 (24GB)	22 (approx.)

K. Analysis of Lower-Body Policy Changes

To clarify the dependency of the Time Optimization Policy (TOP) on the pretrained lower-body controller, we conducted ablation experiments by varying the lower-body policy in two ways: (1) training the lower-body RL policy with a reduced dataset $\mathcal{M}'$ that contains only 50% of the motion clips, and (2) swapping in a lower-body policy pretrained with a different encoder structure E' , which reduces the VAE window size

from 30 to 15 while keeping the latent dimension unchanged at 64 to maintain observation consistency.

TABLE XI: Robustness of the TOP policy under different lower-body controllers and encoders.

Methods	Success Rate (%) $\uparrow$	$E_{jpe}^{upper} \downarrow$	$E_g \downarrow$	$E_{acc}^{upper} \downarrow$
Baseline	95.30	0.0269	2.729	10.41
RL with dataset $\mathcal{M}'$	89.40	0.0338	3.545	10.94
RL with encoder E'	91.85	0.0301	3.312	10.65

These results show that while the TOP can partially adapt to moderate changes in the lower-body dynamics, its performance degrades if the lower-body policy is trained with a reduced dataset or uses a different encoder. Specifically, the success rate drops by approximately 4–6% and the upper-body joint tracking errors increase, indicating that the synergy between the TOP and the lower-body controller benefits from a shared motion prior and consistent encoder design.

This design choice is intentional: by first training a robust lower-body controller with domain randomization and a consistent motion representation, we ensure stable base stabilization under diverse upper-body motions. The TOP then learns to optimize time allocation to best fit the stabilizer’s dynamics.

Although the framework does not assume rigid coupling, it does rely on compatible motion encodings to maintain robust performance. For future work, we plan to investigate end-to-end co-training or meta-adaptation strategies to further reduce this dependency.